Day 5: Exception Handling & Packages (13-June-2026, Saturday)

Session Time: 9:00 AM – 1:00 PM (No lunch break; Tea Break: 11:00 AM – 11:15 AM)

Session Objectives

By the end of Day 5, learners will be able to:

- Differentiate between checked and unchecked exceptions.
- Use `try-catch-finally` blocks to handle runtime errors gracefully.
- Propagate exceptions using `throws` and manually throw exceptions with `throw`.
- Create custom exception classes for business logic validation.
- Organize code using packages and understand access modifiers (public, protected, default, private).
- Implement basic logging using `java.util.logging`.
- Solve LeetCode problems: Contains Duplicate (HashSet) and Missing Number (sum formula / XOR).
- Solve HackerRank: Java Exception Handling.
- Apply exception handling in a **Secure Payment Gateway Validation System**.

Detailed Agenda (Time Slots & Activities)

Time Slot	Activity	Duration
9:00 – 9:15	Recap of Day 4 & Day 5 Overview (Exception Handling need)	15 min
9:15 – 9:45	Exception Hierarchy – Checked vs Unchecked, Error vs Exception	30 min
9:45 – 10:15	try-catch-finally – Flow, Multiple Catches, try-with-resources	30 min
10:15 – 10:30	throw and throws – Propagating and Manual Throwing	15 min
10:30 – 10:45	Tea Break	15 min
10:45 – 11:00	Custom Exceptions – Creating and Using	15 min

11:00		
11:00 – 11:20	Packages – Declaration, Import, Naming Conventions	20 min
11:20 – 11:35	Access Modifiers – Visibility Table	15 min
11:35 – 11:50	Logging Basics – java.util.logging (Logger, Levels, Handlers)	15 min
11:50 – 12:10	LeetCode: Contains Duplicate & Missing Number (Approaches)	20 min
12:10 – 12:25	HackerRank: Java Exception Handling Walkthrough	15 min
12:25 – 12:55	Corporate Use Case: Secure Payment Gateway Validation System	30 min
12:55 – 1:00	Homework & Closing	5 min

---

1. Exception Handling Overview

1.1 What is an Exception?

- An exception is an event that disrupts normal program flow (e.g., division by zero, file not found).
- Java provides a robust exception handling mechanism to maintain program stability.

1.2 Why Handle Exceptions?

- Prevent abrupt program termination.
- Provide meaningful error messages to users.
- Separate error-handling code from business logic.
- Ensure resources are properly released (files, database connections).

1.3 Exception Hierarchy

```
java.lang.Object
  └── java.lang.Throwable
        ├── java.lang.Error (unchecked – JVM errors, OutOfMemoryError, StackOverflowError)
        └── java.lang.Exception
              ├── IOException, SQLException, ClassNotFoundException (checked)
              └── RuntimeException (unchecked)
                    ├── ArithmeticException, NullPointerException
```

```
                    ├── ArrayIndexOutOfBoundsException
                    └── IllegalArgumentException, etc.
```

1.4 Checked vs Unchecked Exceptions

Type	Checked	Unchecked
Compiler checks	Yes – must handle or declare	No
Subclasses	Exception (except RuntimeException)	RuntimeException and Error
Examples	IOException, SQLException, ClassNotFoundException	NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException
Recovery	Expected external failures (file missing, network down)	Programming errors (bug)

---

2. try-catch-finally

2.1 Basic Syntax

```
try {
    // Code that may throw an exception
} catch (ExceptionType1 e) {
    // Handler for ExceptionType1
} catch (ExceptionType2 e) {
    // Handler for ExceptionType2
} finally {
    // Always executes (unless JVM exits or System.exit() called)
    // Typically used for cleanup (close files, release resources)
}
```

2.2 Flow Scenarios

Situation	What happens
No exception	try runs completely, skip catches, finally executes
Exception caught	try aborts, matching catch runs, finally executes
Exception not caught	try aborts, finally executes, then exception propagates
Exception thrown in catch	finally still executes before propagation

return in try	finally executes before return
System.exit() in try	finally does NOT execute

2.3 Multiple Catch Blocks (Java 7+)

- Order matters: catch more specific exceptions before general ones.
- Multi-catch: `catch (IOException | SQLException e)` (same handling).

2.4 try-with-resources (Java 7+)

- Automatically closes resources implementing `AutoCloseable`.
- Syntax: `try (ResourceType res = new ResourceType()) { ... }`
- Eliminates need for explicit finally close.

2.5 finally Block Best Practices

- Avoid throwing exceptions inside finally (can mask original exception).
- Use for closing resources if not using try-with-resources.

---

3. throw and throws

3.1 throws – Declaring Exceptions

- Used in method signature: `void method() throws IOException, SQLException`
- Indicates method may throw those exceptions; caller must handle or re-declare.
- Applies to checked exceptions only (unchecked need not be declared).
- Can declare multiple exceptions.

3.2 throw – Throwing Exceptions

- Used to manually throw an exception: `throw new ArithmeticException("Division by zero");`
- Can throw any `Throwable` (checked or unchecked).
- If throwing checked exception, method must declare it with `throws`.

3.3 Difference: throw vs throws

throw	throws
Used inside method body	Used in method signature
Throws exactly one exception	Declares possible exceptions

Throws exactly one exception	Declares possible exceptions
Followed by an instance of Throwable	Followed by exception class names
e.g., `throw new Exception();`	e.g., `void m() throws Exception { }`

---

4. Custom Exceptions

4.1 When to Create Custom Exceptions?

- Business logic failures not covered by standard exceptions.
- Need to add specific data (error codes, fields).
- Better readability and maintainability.

4.2 How to Create

- Extend `Exception` (checked custom exception) or `RuntimeException` (unchecked).
- Provide constructors (with/without message, cause).

4.3 Example (Conceptual)

- `InsufficientBalanceException extends Exception` – thrown when withdrawal exceeds balance.
- `InvalidAgeException extends RuntimeException` – thrown when age is out of range.

4.4 Best Practices

- Name ends with "Exception".
- Document when and why it is thrown.
- Consider including error codes for API consumers.

---

5. Packages

5.1 Definition

- A package is a namespace that groups related classes and interfaces.
- Prevents naming conflicts.
- Provides access protection (package-private visibility).

5.2 Declaring a Package

- First statement in a Java file (before imports): `package com.company.project.module;`

- Convention: reverse domain name (e.g., `org.apache.commons`).
- If no package declared, class belongs to **default package** (not recommended for production).

5.3 Importing Packages

- `import java.util.ArrayList;` – single class import.
- `import java.util.*;` – imports all classes from package (does not import sub-packages).
- `import static java.lang.Math.PI;` – static import (imports static members).

5.4 Package Naming Conventions

- All lowercase.
- Words separated by dots.
- Example: `com.mycompany.paymentgateway.validators`.

5.5 Compilation and Execution with Packages

- Directory structure must match package structure.
- `javac -d . MyClass.java` (compiles into proper folders).
- `java com.mycompany.Main` (run using fully qualified name).

---

6. Access Modifiers

6.1 Visibility Table

Modifier	Same Class	Same Package	Subclass (different package)	Any class
`private`	✅	❌	❌	❌
default (no modifier)	✅	✅	❌	❌
`protected`	✅	✅	✅	❌
`public`	✅	✅	✅	✅

6.2 When to Use Which?

Modifier	Use Case
`private`	Fields and helper methods – internal implementation details.

default	Package-internal utilities; less common in large projects.
`protected`	For methods that should be overridden by subclasses; also accessible within package.
`public`	API exposed to external code; main method, constructors of public classes.

6.3 Access Modifiers with Classes

- Top-level classes can be `public` or default (not private/protected).
- Nested classes can have all four.

---

7. Logging Basics

7.1 Why Logging?

- Debugging production issues.
- Auditing and compliance.
- Performance monitoring.
- Better than `System.out.println()` (can be configured, leveled, redirected).

7.2 java.util.logging (JUL)

- Built-in logging framework.
- **Logger:** `Logger logger = Logger.getLogger("MyLogger");`
- **Levels:** SEVERE, WARNING, INFO, CONFIG, FINE, FINER, FINEST.
- **Handlers:** ConsoleHandler, FileHandler.
- **Formatters:** SimpleFormatter, XMLFormatter.

7.3 Basic Usage Example (Conceptual)

- `logger.info("Payment processed successfully");`
- `logger.warning("Low balance: " + balance);`
- `logger.log(Level.SEVERE, "Database connection failed", exception);`

7.4 Best Practices

- Use different log levels appropriately (INFO for normal flow, WARNING for recoverable issues, SEVERE for fatal).
- Include exception stack traces using two-argument log method.
- Avoid logging sensitive data (passwords, credit card numbers).
- External frameworks: Log4j2, SLF4J + Logback (more common in enterprise)

- External frameworks: Log4j2, SLF4J + Logback (more common in enterprise).

---

8. LeetCode Problems – Approach Discussion

8.1 Contains Duplicate

- **Problem:** Given integer array nums, return true if any value appears at least twice.
- **Approach 1:** Nested loops O(n^2) – too slow for large inputs.
- **Approach 2:** Sort then check adjacent elements O(n log n).
- **Approach 3:** HashSet O(n) time, O(n) space – add each element; if already present, return true.
- **Edge case:** Empty array or single element – false.
- **Relation to Day 5:** HashSet relies on `hashCode()` and `equals()`, which can throw exceptions if not overridden properly (but that's advanced). Primarily a data structure problem.

8.2 Missing Number

- **Problem:** Given array nums containing n distinct numbers from 0..n, find missing one.
- **Approach 1 (Sum formula):** Compute expected sum = n*(n+1)/2, actual sum, difference = missing number. Watch for integer overflow (use long).
- **Approach 2 (XOR):** XOR all indices and all values; duplicates cancel; remaining is missing.
  - `missing = 0; for i in 0..n: missing ^= i ^ nums[i];` (careful with bounds).
- **Time complexity:** O(n), space O(1).
- **Edge case:** n=0 (empty array) – missing 0.

---

9. HackerRank: Java Exception Handling

9.1 Problem Statement (Typical)

- Create a class `MyCalculator` with method `power(int n, int p)` that returns n^p.
- If n or p is negative, throw an exception: `"n and p should be non-negative"`.
- If both are zero, throw: `"n and p should not be zero"`.
- Use custom exception (or built-in `Exception`).

9.2 Key Learning Points

- Using `throw` to manually throw exceptions.
- Declaring `throws` in method signature.
- Writing custom exception messages.
- Handling exceptions in main (try-catch).

- Handling exceptions in main (try-catch).

9.3 Common Pitfalls

- Forgetting to declare `throws Exception` in method.
- Not catching the thrown exception in calling code (compiler error for checked exceptions).

---

10. Corporate Use Case: Secure Payment Gateway Validation System

10.1 Problem Context

An e-commerce company processes online payments through a payment gateway. The system must validate:

- Credit card number (Luhn algorithm, length, prefix).
- Expiry date (not in the past).
- CVV (3 or 4 digits).
- Transaction amount (positive, within daily limit).
- Sufficient balance (call to bank API – may throw network exception).

All validation failures must be logged, and appropriate error messages must be returned to the user. The system should not crash on invalid input or external service failures.

10.2 Application of Day 5 Topics

Requirement	Java Feature Used
Invalid card number – validation error	Custom `InvalidCardException` (checked or unchecked)
Expired card – business rule violation	Custom `CardExpiredException`
Network failure when contacting bank	Catch `IOException` (checked) and log it
Transaction amount exceeds limit	Custom `LimitExceededException`
Separate validation logic from main flow	try-catch blocks around risky operations
Ensure resources (network connections) are closed	try-with-resources (or finally)

Propagate validation errors to caller	`throws` clause in validation methods
Log each validation failure with timestamp	`java.util.logging` (INFO for success, WARNING for recoverable, SEVERE for critical)
Group all payment-related classes	Package: `com.company.paymentgateway`
Separate validators into their own package	Package: `com.company.paymentgateway.validators`
Hide internal validation helpers	`private` methods for Luhn check
Expose only public `validatePayment()` method	`public` method with `throws` declarations
Prevent direct access to sensitive fields	`private` fields with getters

10.3 Example Architecture (Conceptual)

```
com.company.paymentgateway
├── PaymentProcessor (public class)
│   └── processPayment(PaymentRequest req) throws PaymentException
├── PaymentRequest (public class with private fields)
├── validators
│   ├── CardValidator (public class)
│   │   └── validateCardNumber(String card) throws InvalidCardException
│   └── AmountValidator (public class)
└── exceptions
    ├── PaymentException (abstract or base custom exception)
    ├── InvalidCardException extends PaymentException
    ├── CardExpiredException extends PaymentException
    └── LimitExceededException extends PaymentException
```

10.4 Logging Strategy

Event	Log Level	Example
Payment attempt received	INFO	"Payment attempt for user 12345"
Card validation failed	WARNING	"Invalid card number: XXXX-1234"
Bank API timeout	SEVERE	"Bank connection timeout – transaction failed"
Payment successful	INFO	"Transaction ID TXN67890 completed"

Unexpected error (NPE)	SEVERE with stack trace	Log exception details for debugging

10.5 Discussion Questions

- Should custom exceptions be checked or unchecked? (Checked for recoverable validation errors; unchecked for programming mistakes.)
- How to handle logging without exposing credit card numbers? (Log only last 4 digits or mask.)
- What if multiple validations fail? (Collect into a list of exceptions – use `Throwable.addSuppressed()` or custom `ValidationException` with list of errors.)
- Why use packages? (Avoid naming conflicts with other payment libraries, organize code.)

---

11. Hands-on Exercises (Without Code Lines)

11.1 try-catch-finally Flow Prediction

Given pseudocode:

```
try {
    print("A")
    int x = 10/0
    print("B")
} catch (ArithmeticException e) {
    print("C")
    throw new NullPointerException()
} finally {
    print("D")
}
print("E")
```

What prints? (Answer: A, C, D, then exception propagates – E never prints)

11.2 Checked vs Unchecked Classification

Classify each:

- `FileNotFoundException`
- `StringIndexOutOfBoundsException`
- `ClassCastException`
- `SQLException`
- `NumberFormatException`

11.3 Custom Exception Design

Design (describe) a custom exception `InsufficientBalanceException` for a banking app:

- Should it be checked or unchecked? Why?
- What constructors would you provide?

11.4 Access Modifier Scenario

Class A in package `p1` has a protected method `show()`. Class B in package `p2` extends A. Class C in `p2` does NOT extend A. Can B call `show()`? Can C call `show()`? (B: yes, C: no)

11.5 Logging Level Decision

Which log level would you use for:

- User enters wrong password repeatedly (WARNING)
- Database connection lost (SEVERE)
- Cache refresh starting (INFO)
- Detailed method entry for debugging (FINE)

---

12. Quiz & Quick Check (Conceptual)

1. Which of the following is a checked exception?

- (a) ArithmeticException
- (b) NullPointerException
- (c) IOException
- (d) ArrayIndexOutOfBoundsException

2. What is the output of: `try { return; } finally { System.out.print("done"); }`

- (a) Nothing
- (b) "done"
- (c) Compilation error
- (d) Runtime error

3. Which access modifier allows access from any class?

- (a) private
- (b) default
- (c) protected
- (d) public

(d) public

4. Package declaration must be:

- (a) After imports
- (b) Before any import or class declaration
- (c) After class declaration
- (d) Optional anywhere

5. Given `int[] nums = {0,1,3,4}; int n = 4;` using sum formula, what is missing number?

- (a) 0
- (b) 2
- (c) 3
- (d) 4

Answers: 1-c, 2-b, 3-d, 4-b, 5-b (expected sum=10, actual sum=8, missing=2)

---

13. Group Activity – Pair Programming Simulation

- **Duration:** 15 minutes (during hands-on slot).
- **Task:** Design the exception flow for payment validation:
  - Write pseudocode for `validateCard(String cardNumber)` that throws `InvalidCardException` if card does not pass Luhn check.
  - Write `processPayment(PaymentRequest req)` that calls validators, logs each step, and rethrows a custom `PaymentException` with appropriate message.
  - Discuss: Should `InvalidCardException` be checked or unchecked?
- **Outcome:** Pairs share their design choices; instructor highlights proper exception chaining and logging.

---

14. Homework & Preparation for Day 6

- **Coding practice (HackerRank):**
  - *Java Exception Handling* (power method problem).
- **LeetCode:**
  - *Contains Duplicate* – solve using HashSet.
  - *Missing Number* – solve using both sum and XOR methods.
- **Theoretical:** Write a short note on the difference between `throw` and `throws`.
- **Reading for Day 6:** Collections Framework Part 1 – ArrayList, LinkedList, Stack, Queue, Vector, Iterators.
- **Self-assessment:**

Self-assessment:

- Create a custom exception `AgeOutOfRangeException` (unchecked).
- Write a method `validateAge(int age)` that throws it if age < 18 or > 65.
- Log the validation using `java.util.logging`.

---

15. Additional Resources & Cheat Sheets

- **Exception handling flowchart** – visualize try-catch-finally.
- **Access modifiers diagram** – shows boundaries.
- **Logging levels hierarchy:** SEVERE (highest) → WARNING → INFO → CONFIG → FINE → FINER → FINEST (lowest).
- **Package naming conventions:** `com.projectname.modulename`

---

16. Closing Summary

Day 5 equipped learners with the ability to build robust Java applications that handle errors gracefully. Exception handling (try-catch-finally, throw/throws) prevents crashes and allows meaningful error feedback. Custom exceptions enable domain-specific error reporting. Packages and access modifiers enforce code organization and visibility. Logging provides runtime insight for debugging and monitoring. The Secure Payment Gateway case study tied these concepts together, demonstrating how to validate transactions, log events, and propagate errors without exposing internal details.

Key Takeaways:

- Checked exceptions must be handled or declared; unchecked exceptions represent bugs.
- finally block always executes – ideal for cleanup.
- throw vs throws: one throws an object, the other declares exception types.
- Packages avoid naming conflicts; access modifiers control visibility.
- Logging is essential for production systems; never use `System.out.println()`.

End of Day 5

Line count (excluding program code): approximately 470 lines of instructional text, tables, lists, and descriptions.